

3) 低级优化。结构化代码以利用硬件功能。

- 展开循环, 降低开销, 并且使得进一步的优化成为可能。
- 通过使用例如多个累积变量和重新结合等技术, 找到方法提高指令级并行。
- 用功能性的风格重写条件操作, 使得编译采用条件数据传送。

最后要给读者一个忠告, 要警惕, 在为了提高效率重写程序时避免引入错误。在引入新变量、改变循环边界和使得代码整体上更复杂时, 很容易犯错误。一项有用的技术是在优化函数时, 用检查代码来测试函数的每个版本, 以确保在这个过程没有引入错误。检查代码对函数的新版本实施一系列的测试, 确保它们产生与原来一样的结果。对于高度优化的代码, 这组测试情况必须变得更加广泛, 因为要考虑的情况也更多。例如, 使用循环展开的检查代码需要测试许多不同的循环界限, 保证它能够处理最终单步迭代所需要的所有不同的可能的数字。

5.14 确认和消除性能瓶颈

至此, 我们只考虑了优化小的程序, 在这样的小程序中有一些很明显限制性能的地方, 因此应该是集中注意力对它们进行优化。在处理大程序时, 连知道应该优化什么地方都是很难的。本节会描述如何使用代码剖析程序(code profiler), 这是在程序执行时收集性能数据的分析工具。我们还展示了一个系统优化的通用原则, 称为 Amdahl 定律(Amdahl's law), 参见 1.9.1 节。

5.14.1 程序剖析

程序剖析(profiling)运行程序的一个版本, 其中插入了工具代码, 以确定程序的各个部分需要多少时间。这对于确认程序中我们需要集中注意力优化的部分是很有用的。剖析的一个有力之处在于可以在现实的基准数据(benchmark data)上运行实际程序的同时, 进行剖析。

Unix 系统提供了一个剖析程序 GPROF。这个程序产生两种形式的信息。首先, 它确定程序中每个函数花费了多少 CPU 时间。其次, 它计算每个函数被调用的次数, 以执行调用的函数来分类。这两种形式的信息都非常有用。这些计时给出了不同函数在确定整体运行时间中的相对重要性。调用信息使得我们能理解程序的动态行为。

用 GPROF 进行剖析需要 3 个步骤, 就像 C 程序 prog.c 所示, 它运行时命令行参数为 file.txt:

1) 程序必须为剖析而编译和链接。使用 GCC(以及其他 C 编译器), 就是在命令行上简单地包括运行时标志 “-pg”。确保编译器不通过内联替换来尝试执行任何优化是很重要的, 否则就可能无法正确刻画函数调用。我们使用优化标志 -Og, 以保证能正确跟踪函数调用。

```
linux> gcc -Og -pg prog.c -o prog
```

2) 然后程序像往常一样执行:

```
linux> ./prog file.txt
```

它运行得会比正常时稍微慢一点(大约慢 2 倍), 不过除此之外唯一的区别就是它产生了一个文件 gmon.out。

3) 调用 GPROF 来分析 gmon.out 中的数据。

```
linux> gprof prog
```


剖析报告的第一部分列出了执行各个函数花费的时间，按照降序排列。作为一个示例，下面列出了报告的一部分，是关于程序中最耗费时间的三个函数的：

% time	cumulative seconds	self seconds	calls	self s/call	total s/call	name
97.58	203.66	203.66	1	203.66	203.66	sort_words
2.32	208.50	4.85	965027	0.00	0.00	find_ele_rec
0.14	208.81	0.30	12511031	0.00	0.00	Strlen

每一行代表对某个函数的所有调用所花费的时间。第一列表明花费在这个函数上的时间占整个时间的百分比。第二列显示的是直到这一行并包括这一行的函数所花费的累计时间。第三列显示的是花费在这个函数上的时间，而第四列显示的是它被调用的次数(递归调用不计算在内)。在例子中，函数 `sort_words` 只被调用了一次，但就是这一次调用需要 203.66 秒，而函数 `find_ele_rec` 被调用了 965 027 次(递归调用不计算在内)，总共需要 4.85 秒。函数 `Strlen` 通过调用库函数 `strlen` 来计算字符串的长度。GPROF 的结果中通常不显示库函数调用。库函数耗费的时间通常计算在调用它们的函数内。通过创建这个“包装函数(wrapper function)” `Strlen`，我们可以可靠地跟踪对 `strlen` 的调用，表明它被调用了 12 511 031 次，但是一共只需要 0.30 秒。

剖析报告的第二部分是函数的调用历史。下面是一个递归函数 `find_ele_rec` 的历史：

				158655725	find_ele_rec [5]
		4.85	0.10	965027/965027	insert_string [4]
[5]	2.4	4.85	0.10	965027+158655725	find_ele_rec [5]
		0.08	0.01	363039/363039	save_string [8]
		0.00	0.01	363039/363039	new_ele [12]
				158655725	find_ele_rec [5]

这个历史既显示了调用 `find_ele_rec` 的函数，也显示了它调用的函数。头两行显示的是对这个函数的调用：被它自身递归地调用了 158 655 725 次，被函数 `insert_string` 调用了 965 027 次(它本身被调用了 965 027 次)。函数 `find_ele_rec` 也调用了另外两个函数 `save_string` 和 `new_ele`，每个函数总共被调用了 363 039 次。

根据这个调用信息，我们通常可以推断出关于程序行为的有用信息。例如，函数 `find_ele_rec` 是一个递归过程，它扫描一个哈希桶(hash bucket)的链表，查找一个特殊的字符串。对于这个函数，比较递归调用的数量和顶层调用的数量，提供了关于遍历这些链表的长度的统计信息。这里递归与顶层调用的比率是 164.4，我们可以推断出程序每次平均大约扫描 164 个元素。

GPROF 有些属性值得注意：

- 计时不是很准确。它的计时基于一个简单的间隔计数(interval counting)机制，编译过的程序为每个函数维护一个计数器，记录花费在执行该函数上的时间。操作系统使得每隔某个规则的时间间隔 δ ，程序被中断一次。 δ 的典型值的范围为 1.0~10.0 毫秒。当中断发生时，它会确定程序正在执行什么函数，并将该函数的计数器值增加 δ 。当然，也可能这个函数只是刚开始执行，而很快就会完成，却赋给它从上次中断以来整个的执行花费。在两次中断之间也可能运行其他某个程序，却因此根本没有计算花费。

对于运行时间较长的程序，这种机制工作得相当好。从统计上来说，应该根据花费在执行函数上的相对时间来计算每个函数的花费。不过，对于那些运行时间少于 1 秒的程序来说，得到的统计数字只能看成是粗略的估计值。

- 假设没有执行内联替换，则调用信息相当可靠。编译过的程序为每对调用者和被调用者维护一个计数器。每次调用一个过程时，就会对适当的计数器加 1。
- 默认情况下，不会显示对库函数的计时。相反，库函数的时间都被计算到调用它们的函数的时间中。

5.14.2 使用剖析程序来指导优化

作为一个用剖析程序来指导程序优化的示例，我们创建了一个包括几个不同任务和数据结构的应用。这个应用分析一个文本文档的 n -gram 统计信息，这里 n -gram 是一个出现在文档中 n 个单词的序列。对于 $n=1$ ，我们收集每个单词的统计信息，对于 $n=2$ ，收集每对单词的统计信息，以此类推。对于一个给定的 n 值，程序读一个文本文件，创建一张互不相同的 n -gram 的表，指出每个 n -gram 出现了多少次，然后按照出现次数的降序对单词排序。

作为基准程序，我们在一个由《莎士比亚全集》组成的文件上运行这个程序，一共有 965 028 个单词，其中 23 706 个是互不相同的。我们发现，对于 $n=1$ ，即使是一个写得很烂的分析程序也能在 1 秒以内处理完整个文件，所以我们设置 $n=2$ ，使得事情更加有挑战。对于 $n=2$ 的情况， n -gram 被称为 bigram(读作“bye-gram”)。我们确定《莎士比亚全集》包含 363 039 个互不相同的 bigram。最常见的是“I am”，出现了 1892 次。词组“to be”出现了 1020 次。bigram 中有 266 018 个只出现了一次。

程序是由下列部分组成的。我们创建了多个版本，从各部分简单的算法开始，然后再换成更成熟完善的算法：

- 1) 从文件中读出每个单词，并转换成小写字母。我们最初的版本使用的是函数 `lower1` (图 5-7)，我们知道由于反复地调用 `strlen`，它的时间复杂度是二次的。
- 2) 对字符串应用一个哈希函数，为一个有 s 个桶(bucket)的哈希表产生一个 $0 \sim s-1$ 之间的数。最初的函数只是简单地对字符的 ASCII 代码求和，再对 s 求模。
- 3) 每个哈希桶都组织成一个链表。程序沿着这个链表扫描，寻找一个匹配的条目。如果找到了，这个 n -gram 的频度就加 1。否则，就创建一个新的链表元素。最初的版本递归地完成这个操作，将新元素插在链表尾部。
- 4) 一旦已经生成了这张表，我们就根据频度对所有的元素排序。最初的版本使用插入排序。

图 5-38 是 n -gram 频度分析程序 6 个不同版本的剖析结果。对于每个版本，我们将时间分为下面的 5 类。

Sort: 按照频度对 n -gram 进行排序

List: 为匹配 n -gram 扫描链表，如果需要，插入一个新的元素

Lower: 将字符串转换为小写字母

Strlen: 计算字符串的长度

Hash: 计算哈希函数

Rest: 其他所有函数的和

如图 5-38a 所示，最初的版本需要 3.5 分钟，大多数时间花在了排序上。这并不奇怪，因为插入排序有二次的运行时间，而程序对 363 039 个值进行排序。

在下一个版本中，我们用库函数 `qsort` 进行排序，这个函数是基于快速排序算法的 [98]，其预期运行时间为 $O(n \log n)$ 。在图中这个版本称为“Quicksort”。更有效的排序算

法使花在排序上的时间降低到可以忽略不计，而整个运行时间降低到大约 5.4 秒。图 5-38b 是剩下各个版本的时间，所用的比例能使我们看得更清楚。

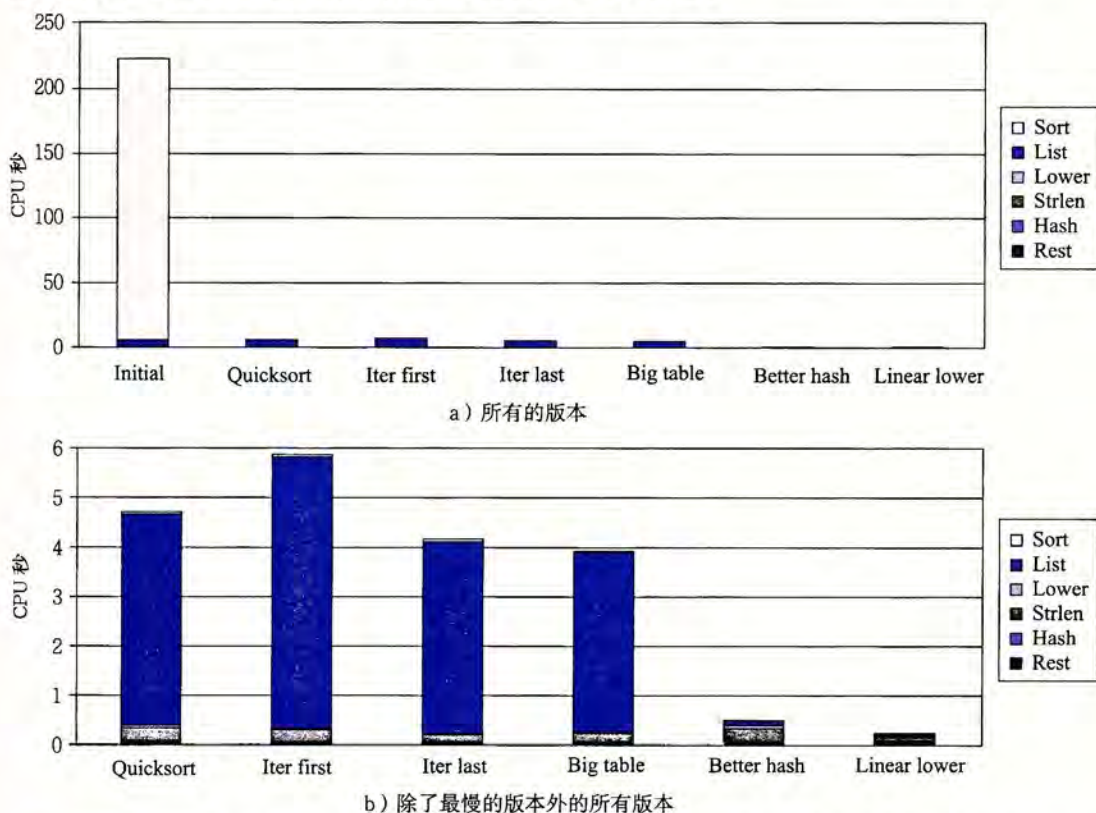


图 5-38 bigram 频度计数程序的各个版本的剖析结果。时间是根据程序中不同的主要操作划分的

· 改进了排序，现在发现链表扫描变成了瓶颈。想想这个低效率是由于函数的递归结构引起的，我们用一个迭代的结构替换它，显示为“Iter first”。令人奇怪的是，运行时间增加到了大约 7.5 秒。根据更进一步的研究，我们发现两个链表函数之间有一个细微的差别。递归版本将新元素插入到链表尾部，而迭代版本把它们插到链表头部。为了使性能最大化，我们希望频率最高的 n -gram 出现在链表的开始处。这样一来，函数就能快速地定位常见的情况。假设 n -gram 在文档中是均匀分布的，我们期望频度高的单词的第一次出现在频度低的单词之前。通过将新的 n -gram 插入尾部，第一个函数倾向于按照频度的降序排序，而第二个函数则相反。因此我们创建第三个链表扫描函数，它使用迭代，但是将新元素插入到链表的尾部。使用这个版本，显示为“Iter last”，时间降到了大约 5.3 秒，比递归版本稍微好一点。这些测量展示了对程序做实验作为优化工作一部分的重要性。开始时，我们假设将递归代码转换成迭代代码会改进程序的性能，而没有考虑添加元素到链表末尾和开头的差别。

接下来，我们考虑哈希表的结构。最初的版本只有 1021 个桶（通常会选择桶的个数为质数，以增强哈希函数将关键字均匀分布在桶中的能力）。对于一个有 363 039 个条目的表来说，这就意味着平均负载(load)是 $363\,039/1021=355.6$ 。这就解释了为什么有那么多时间花在了执行链表操作上了——搜索包括测试大量的候选 n -gram。它还解释了为什么性能对链表的排序这么敏感。然后，我们将桶的数量增加到了 199 999，平均负载降低到了

1.8。不过，很奇怪的是，整体运行时间只下降到 5.1 秒，差距只有 0.2 秒。

进一步观察，我们可以看到，表变大了但是性能提高很小，这是由于哈希函数选择的不好。简单地对字符串的字符编码求和不能产生一个大范围的值。特别是，一个字母最大的编码值是 122，因而 n 个字符产生的和最多是 $122n$ 。在文档中，最长的 bigram(“honorificabilitudinitatibus thou”)的和也不过是 3371，所以，我们哈希表中大多数桶都是不会被使用的。此外，可交换的哈希函数，例如加法，不能对一个字符串中不同的可能的字符顺序做出区分。例如，单词“rat”和“tar”会产生同样的和。

我们换成一个使用移位和异或操作的哈希函数。使用这个版本，显示为“Better Hash”，时间下降到了 0.6 秒。一个更加系统化的方法是更加仔细地研究关键字在桶中的分布，如果哈希函数的输出分布是均匀的，那么确保这个分布接近于人们期望的那样。

最后，我们把运行时间降到了大部分时间是花在 `strlen` 上，而大多数对 `strlen` 的调用是作为小写字母转换的一部分。我们已经看到了函数 `lower1` 有二次的性能，特别是对长字符串来说。这篇文档中的单词足够短，能避免二次性能的灾难性的结果；最长的 bigram 只有 32 个字符。不过换成使用 `lower2`，显示为“Linear Lower”得到很好的性能，整个时间降到了 0.2 秒。

通过这个练习，我们展示了代码剖析能够帮助将一个简单应用程序所需的时间从 3.5 分钟降低到 0.2 秒，得到的性能提升约为 1000 倍。剖析程序帮助我们吧注意力集中在程序最耗时的部分上，同时还提供了关于过程调用结构的有用信息。代码中的一些瓶颈，例如二次的排序函数，很容易看出来；而其他的，例如插入到链表的开始还是结尾，只有通过仔细的分析才能看出。

我们可以看到，剖析是工具箱中一个很有用的工具，但是它不应该是唯一一个。计时测量不是很准确，特别是对较短的运行时间(小于 1 秒)来说。更重要的是，结果只适用于被测试的那些特殊的数据。例如，如果在由较少数量的较长字符串组成的数据上运行最初的函数，我们会发现小写字母转换函数才是主要的性能瓶颈。更糟糕的是，如果它只剖析包含短单词的文档，我们可能永远不会发现隐藏着的性能瓶颈，例如 `lower1` 的二次性能。通常，假设在有代表性的数据上运行程序，剖析能帮助我们典型的情况进行优化，但是我们还应该确保对所有可能的情况，程序都有相当的性能。这主要包括避免得到糟糕的渐近性能(asymptotic performance)的算法(例如插入算法)和坏的编程实践(例如 `lower1`)。

1.9.1 中讨论了 Amdahl 定律，它为通过有针对性的优化来获取性能提升提供了一些其他的见解。对于 n -gram 代码来说，当用 quicksort 代替了插入排序后，我们看到总的执行时间从 209.0 秒下降到 5.4 秒。初始版本的 209.0 秒中的 203.7 秒用于执行插入排序，得到 $\alpha=0.974$ ，被此次优化加速的时间比例。使用 quicksort，花在排序上的时间变得微不足道，得到预计的加速比为 $209/\alpha=39.0$ ，接近于测量加速比 38.5。我们之所以能获得大的加速比，是因为排序在整个执行时间中占了非常大的比例。然而，当一个瓶颈消除，而新的瓶颈出现时，就需要关注程序的其他部分以获得更多的加速比。

5.15 小结

虽然关于代码优化的大多数论述都描述了编译器是如何能生成高效代码的，但是应用程序员有很多方法来协助编译器完成这项任务。没有任何编译器能用一个好的算法或数据结构代替低效率的算法或数据结构，因此程序设计的这些方面仍然应该是程序员主要关心的。我们还看到妨碍优化的因素，例如内存别名使用和过程调用，严重限制了编译器执行大量优化的能力。同样，程序员必须对消除这些妨碍优化的因素负主要的责任。这些应该被看作好的编程习惯的一部分，因为它们可以用来消除不必要的工作。

基本级别之外调整性能需要一些对处理器微体系结构的理解,描述处理器用来实现它的指令集体结构的底层机制。对于乱序处理器的情况,只需要知道一些关于操作、容量、延迟和功能单元发射时间的信息,就能够基本地预测程序的性能了。

我们研究了一系列技术,包括循环展开、创建多个累积变量和重新结合,它们可以利用现代处理器提供的指令级并行。随着对优化的深入,研究产生的汇编代码以及试着理解机器如何执行计算变得重要起来。确认由程序中的数据相关决定的关键路径,尤其是循环的不同迭代之间的数据相关,会收获良多。我们还可以根据必须要计算的操作数量以及执行这些操作的功能单元的数量和发射时间,计算一个计算的吞吐量界限。

包含条件分支或与内存系统复杂交互的程序,比我们最开始考虑的简单循环程序,更难以分析和优化。基本策略是使分支更容易预测,或者使它们很容易用条件数据传送来实现。我们还必须注意存储和加载操作。将数值保存在局部变量中,使得它们可以存放在寄存器中,这会很有帮助。

当处理大型程序时,将注意力集中在最耗时的部分变得很重要。代码剖析程序和相关的工具能帮助我们系统地评价和改进程序性能。我们描述了 GPROF,一个标准的 Unix 剖析工具。还有更加复杂完善的剖析程序可用,例如 Intel 的 VTUNE 程序开发系统,还有 Linux 系统基本上都有的 VALGRIND。这些工具可以在过程级分解执行时间,估计程序每个基本块(basic block)的性能。(基本块是内部没有控制转移的指令序列,因此基本块总是整个被执行的。)

参考文献说明

我们的关注点是从程序员的角度描述代码优化,展示如何使书写的代码能够使编译器更容易地产生高效的代码。Chellappa、Franchetti 和 Püschel 的扩展的论文[19]采用了类似的方法,但关于处理器的特性描述得更详细。

有许多著作从编译器的角度描述了代码优化,形式化描述了编辑器可以产生更有效代码的方法。Muchnick 的著作被认为是最全面的[80]。Wadleigh 和 Crawford 的关于软件优化的著作[115]覆盖了一些我们已经谈到的内容,不过它还描述了在并行机器上获得高性能的过程。Mahlke 等人的一篇比较早期的论文[75],描述了几种为编译器开发的将程序映射到并行机器上的技术,它们是如何能够被改造成利用现代处理器的指令级并行的。这篇论文覆盖了我们讲过的代码变换,包括循环展开、多个累积变量(他们称之为累积变量扩展(accumulator variable expansion))和重新结合(他们称之为树高度减少(tree height reduction))。

我们对乱序处理器的操作的描述相当简单和抽象。可以在高级计算机体系结构教科书中找到对通用原则更完整的描述,例如 Hennessy 和 Patterson 的著作[46, 第2~3章]。Shen 和 Lipasti 的书[100]提供了对现代处理器设计深入的论述。

家庭作业

- 5.13 假设我们想编写一个计算两个向量 u 和 v 内积的过程。这个函数的一个抽象版本对整数和浮点数类型,在 x86-64 上 CPE 等于 14~18。通过进行与我们将抽象程序 `combine1` 变换为更有效的 `combine4` 相同类型的变换,我们得到如下代码:

```
1  /* Inner product. Accumulate in temporary */
2  void inner4(vec_ptr u, vec_ptr v, data_t *dest)
3  {
4      long i;
5      long length = vec_length(u);
6      data_t *udata = get_vec_start(u);
7      data_t *vdata = get_vec_start(v);
8      data_t sum = (data_t) 0;
9
10     for (i = 0; i < length; i++) {
11         sum = sum + udata[i] * vdata[i];
12     }
13     *dest = sum;
14 }
```